Gestão de Alojamentos Turísticos

Hugo Manuel Alves Gomes

Nº 31545 – Regime Pós-laboral

Ano letivo 2025/2026

Docente: Ernesto Casanova

Unidade Curricular - Programação Orientada a Objetos

Licenciatura em Engenharia de Sistemas Informáticos

Escola Superior de Tecnologia

Instituto Politécnico do Cávado e do Ave

Índice

Índice de figuras 1

Resumo 2

Glossário 3

1. Introdução 4

1.1 Estrutura do documento 4

1.2 Motivação 5

1.3 Enquadramento 6

1.4 Objetivos 6

2. Análise e Especificação 7

2.1 Requisitos 7

2.1.1 Listas 7

3. Princípios Fundamentais de Programação Orientada a Objetos 9

3.2 Os Quatro Pilares Fundamentais 9

4. Diagrama de Classes UML 11

4.1 Estrutura do Diagrama 11

4.2 Elementos Principais Representados 11

4.3 Valor para o Desenvolvimento 12

5. Modelação das Classes Principais 13

5.1 Classes Principais Implementadas 13

5.2 Relações e Colaborações 15

6. Estratégia de Qualidade e Robustez 16

7. Estratégia de Persistência 17

7.1 Vantagens Técnicas 17

7.2. Otimizações e Considerações Críticas 18

8. Enquadramento Arquitetural: Padrão Model-View-ViewModel (MVVM) 19

8.1 Componentes e Responsabilidades 19

8.2 Vantagens Estratégicas e Técnicas do MVVM 20

8.3 Fluxo de Dados e Controlo 21

9. Requisitos do Sistema 22

9.1 Requisitos funcionais (O que o sistema faz): 22

9.2 Requisitos Não Funcionais (Qualidade do sistema): 23
10. Exceções Personalizadas: A Classe BusinessException 24
11. Validação de Segurança via DLL Externa 25
12. Conclusão e Trabalho Futuro 26
12.1 Dificuldades Observadas 27
12.2 Trabalho Futuro 28
12.3 GitHub Pessoal: 28
13. Referências bibliográficas 29

Índice de figuras

Figura 1 Diagrama de Classes UML do Sistema de Gestão de Alojamentos Turísticos.. 12
Figura 2 Representação de MVVM..... 19

Resumo

Nesta primeira fase do trabalho, iniciou-se o desenvolvimento de uma aplicação em C# com base no paradigma Programação Orientada a Objetos (POO), seguindo a arquitetura MVVM e utilizando persistência em JSON.

O objetivo é criar um sistema para Gestão de Alojamentos Turísticos, capaz de gerir clientes, alojamentos, reservas, check-ins, consultas e registos, através de uma estrutura modular e escalável.

Foram identificadas e implementadas as classes principais do domínio: Cliente, Alojamento, Reserva, CheckIn, Consulta, Registo.

Cada classe foi definida com os seus atributos e métodos essenciais, aplicando os pilares da POO (encapsulamento, abstração, polimorfismo e herança). Além disso, foi planeada a camada de persistência através de serialização e deserialização em JSON, permitindo guardar e carregar dados de forma simples e eficiente.

A arquitetura MVVM foi delineada para garantir separação de responsabilidades:

Model: entidades e lógica de negócio.

ViewModel: propriedades observáveis e comandos para interação com a UI.

View: interfaces gráficas em WPF com data binding.

Esta fase inclui também a elaboração do diagrama UML das classes e a definição da estrutura do projeto, preparando a base para a implementação completa na segunda fase.

Glossário

Alojamento - Unidade de hospedagem disponível para reserva, podendo ser do tipo hotel, apartamento ou quarto privado.

Check-in - Processo de entrada dos clientes nos alojamentos.

Cliente - Hóspede ou utilizador do sistema que efetua reservas nos alojamentos turísticos.

Consulta - Operação de pesquisa e filtragem de dados no sistema para recuperação de informação específica.

Data Binding - Mecanismo do WPF que permite ligação automática e declarativa de dados entre a View e o ViewModel.

Encapsulamento - Princípio da POO que agrupa dados e comportamentos numa classe, ocultando detalhes de implementação.

Herança - Mecanismo da POO que permite que uma classe adquira características de outra classe, promovendo reutilização de código.

JSON - Formato de dados leve utilizado para persistência no sistema.

MVVM - Padrão arquitetural Model-View-ViewModel que promove separação de responsabilidades entre interface e lógica de negócio.

ObservableCollection - Estrutura de dados do .NET que notifica automaticamente a interface sobre alterações na coleção.

Polimorfismo - Capacidade de objetos de diferentes classes responderem ao mesmo método de maneiras específicas.

Registo - Entidade que atua como agregadora de operações de criação e atualização no sistema.

Reserva - Relação entre clientes e alojamentos que coordena todo o processo de marcação e estadia.

WPF - Windows Presentation Foundation, tecnologia da Microsoft para desenvolvimento de interfaces gráficas.

XAML - Linguagem de marcação utilizada para definir interfaces de utilizador em aplicações WPF.

LINQ - Language Integrated Query, tecnologia que permite escrever consultas diretamente em código C#.

Repository Pattern - Padrão de design que abstrai a camada de acesso a dados, centralizando operações de persistência.

Command Pattern - Padrão comportamental que encapsula solicitações como objetos, permitindo parametrizar clientes com diferentes pedidos.

Serialização - Processo de conversão de objetos em formato adequado para armazenamento ou transmissão.

Testes Unitários - Prática de desenvolvimento que valida o comportamento isolado de componentes individuais do sistema.

1. Introdução

No presente capítulo pretende-se contextualizar o trabalho desenvolvido, a motivação da sua origem, objetivos traçados, enquadramento académico e a estrutura geral do documento, de forma a fornecer uma visão clara do percurso realizado ao longo do projeto.

1.1 Estrutura do documento

Capítulo 1 - Introdução

Capítulo 2 - Análise e Especificação de Requisitos

Capítulo 3 - Arquitetura e Design do Sistema

Capítulo 4 - Implementação

Capítulo 5 - Conclusão e Futuras Melhorias

Capítulo 6 - Referências Bibliográficas

1.2 Motivação

A gestão eficiente de alojamentos turísticos em Portugal constitui um desafio crescente no atual panorama económico, particularmente face à expansão significativa do sector turístico nacional. Um dos principais problemas identificados neste domínio consiste na fragmentação de sistemas e processos manuais para gerir as diversas operações inerentes à administração de estabelecimentos de hospedagem.

A realidade atual demonstra que muitos empreendimentos turísticos de pequena e média dimensão ainda dependem de soluções desconexas para administrar as diferentes vertentes do seu negócio. Esta desorganização manifesta-se através de:

- Processos manuais na gestão de reservas e check-ins, propensos a erros;
- Dificuldade em obter informação consolidada sobre ocupação, receitas e desempenho operacional;
- Falta de sincronização entre a disponibilidade real e as diferentes plataformas de reserva;

Apesar da crescente necessidade de soluções integradas para a automatização destes processos, observa-se ainda uma escassez de ferramentas adaptadas à realidade específica dos alojamentos turísticos portugueses, que sejam simultaneamente acessíveis e flexíveis.

A pertinência deste trabalho insere-se no contexto de transformação digital do sector turístico, que exige ferramentas robustas e integradas para competir num mercado global em constante evolução.

1.3 Enquadramento

Este trabalho foi desenvolvido no âmbito da Unidade Curricular de Programação Orientada a Objetos do curso de Licenciatura em Engenharia de Sistemas Informáticos, com o objetivo de aplicar os conhecimentos teóricos adquiridos durante o semestre na resolução de um problema prático relacionado com a gestão de alojamentos turísticos.

O projeto enquadra-se na temática de desenvolvimento de sistemas para negócios do sector turístico, permitindo a consolidação de competências em análise, design e implementação de software seguindo o paradigma orientado a objetos.

1.4 Objetivos

Os principais objetivos deste projeto são:

- Desenvolver uma solução de software integrada para gestão de alojamentos turísticos, abrangendo todos os processos operacionais;
- Implementar uma arquitetura robusta baseada no padrão MVVM, promovendo a separação de responsabilidades e a manutenibilidade do código;
- Criar um sistema de persistência eficiente para armazenamento de dados;
- Desenvolver funcionalidades completas para registo, consulta, reservas, check-in e gestão de clientes e alojamentos;
- Aplicar os princípios fundamentais da programação orientada a objetos: encapsulamento, herança, polimorfismo e abstração;
- Implementar testes unitários com cobertura mínima de 50%, garantindo a qualidade e robustez do código;
- Utilizar estruturas de dados adequadas para as operações do sistema;
- Aplicar padrões de design estabelecidos como Repository e Command Pattern;
- Utilizar funções como LINQ, promovendo código mais expressivo e conciso.

2. Análise e Especificação

Neste capítulo pretende-se demonstrar os requisitos, arquiteturas lógicas/funcionais, modelação e interação necessários para o funcionamento do projeto desenvolvido.

2.1 Requisitos

Neste subcapítulo apresentam-se alguns dos requisitos necessários para a realização deste trabalho.

2.1.1 Listas

A escolha das listas genéricas (List<T>) como estrutura de dados fundamental para este projeto foi cuidadosamente ponderada, considerando as exigências específicas de um sistema de gestão de alojamentos turísticos. Esta opção justifica-se pela necessidade de gerir conjuntos de dados dinâmicos, onde a flexibilidade na administração de coleções e a eficiência na execução de operações de consulta e manipulação são fatores críticos para o sucesso da aplicação.

Conceptualmente, uma lista representa uma estrutura de dados linear e ordenada que possibilita o armazenamento sequencial de elementos. Esta estrutura diferencia-se dos arrays tradicionais pela sua capacidade de redimensionamento automático e pela sua eficiência.

No âmbito deste projeto de desenvolvimento, estão previstas a implementação de múltiplas listas especializadas para as diferentes vertentes do sistema. Entre estas destacam-se a Lista de Alojamentos, responsável pelo registo de todas as unidades de hospedagem a Lista de Clientes, que gere a informação dos hóspedes e a Lista de Reservas, que controla todo o processo de marcações e estadias.

A decisão de utilizar listas genéricas, em detrimento de outras estruturas como arrays de dimensão fixa ou listas ligadas, fundamenta-se em quatro pilares principais:

Em primeiro lugar, a Natureza Dinâmica dos Dados do domínio da gestão hoteleira, onde se assume que o número de alojamentos, clientes e reservas sofre variações constantes ao longo do tempo, exigindo uma estrutura de dados capaz de se adaptar a estas flutuações.

Em segundo lugar, a necessidade de suportar Operações Frequentes de Consulta complexas, que envolvem pesquisas e filtragens avançadas, operações estas que são facilitadas pela integração natural das listas com tecnologias como LINQ (Language Integrated Query).

O terceiro pilar reside na Integração com Binding WPF, uma vez que as listas oferecem compatibilidade total com ObservableCollection, o que permite atualizações em tempo real da interface gráfica sempre que os dados subjacentes sofrem modificações.

Finalmente, considera-se que as listas proporcionam Performance Adequada para o volume de dados esperado neste contexto específico de aplicação, assegurando tempos de resposta apropriados para as operações mais críticas do sistema.

A adoção de listas genéricas estabelece assim uma base sólida e eficiente para a implementação do sistema de gestão, garantindo simultaneamente a flexibilidade necessária para suportar operações complexas de negócio e a escalabilidade requerida para acomodar o crescimento futuro da aplicação.

3. Princípios Fundamentais de Programação Orientada a Objetos

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento de software que se baseia no conceito fundamental de "objetos". Estes objetos são instâncias que combinam dados (propriedades) e código (métodos) numa única entidade. A arquitetura do sistema de gestão de alojamentos turísticos foi desenvolvida com base nos quatro pilares essenciais da POO, visando a criação de uma solução robusta, escalável e de elevada qualidade técnica.

3.2 Os Quatro Pilares Fundamentais

A implementação da solução foi guiada pela aplicação dos princípios da POO, conforme descrito nos pontos seguintes.

Encapsulamento

O Encapsulamento consiste em agrupar dados e comportamentos numa unidade (a classe), ocultando os detalhes de implementação e expondo apenas uma interface controlada para a manipulação dos dados internos.

Este princípio foi aplicado para garantir que o estado interno dos objetos seja consistente, protegendo os dados de modificações externas não autorizadas e centralizando a lógica de controlo de acesso e validação.

Herança

A Herança permite que uma classe (subclasse) adquira as características (propriedades e métodos) de outra classe (superclasse). É o mecanismo primário para a reutilização de código e para a organização de classes numa estrutura hierárquica.

A herança foi utilizada para evitar a duplicação de código comum entre entidades e para estabelecer uma hierarquia de classes lógica, facilitando a especialização de funcionalidades a partir de um comportamento base.

Polimorfismo

O Polimorfismo é a capacidade de objetos de diferentes classes responderem ao mesmo método de maneiras específicas e distintas. Promove a flexibilidade e a generalização na forma como o código interage com diferentes tipos de dados.

Este princípio foi fundamental para a conceção de componentes genéricos e intercambiáveis, permitindo que a mesma chamada de método gere comportamentos distintos conforme o objeto que a executa.

Abstração

A Abstração foca-se na apresentação apenas dos aspetos essenciais de um objeto ou sistema, omitindo os detalhes de implementação irrelevantes. É a base para a gestão da complexidade através da definição de fronteiras claras.

A abstração foi aplicada na modelação conceptual das entidades e na definição de contratos de serviço (interfaces), o que permitiu concentrar o desenvolvimento na lógica de negócio central.

A aplicação rigorosa dos princípios fundamentais de Programação Orientada a Objetos constitui a base sólida para o desenvolvimento do sistema de gestão de alojamentos turísticos. Esta abordagem assegura não apenas a qualidade técnica da solução, mas também a sua capacidade de evolução face a requisitos futuros, garantindo a sustentabilidade a longo prazo do projeto.

4. Diagrama de Classes UML

O diagrama de classes UML, desenvolvido utilizando o Visual Paradigm, apresenta de forma gráfica a estrutura completa do sistema e as relações entre as diversas entidades que compõem o domínio de gestão de alojamentos turísticos. Esta representação visual permite uma compreensão imediata da complexidade do sistema e das interações entre os diferentes componentes.

4.1 Estrutura do Diagrama

O diagrama elaborado organiza as classes em pacotes lógicos que refletem a arquitetura em camadas do sistema, destacando-se:

Entidades de Domínio: Representam os conceitos fundamentais do negócio, incluindo as classes Alojamento, Cliente, Reserva e CheckIn;

Relações de Associação: Ilustram as conexões entre as entidades, como a relação entre Cliente e Reserva, e entre Reserva e Alojamento.

4.2 Elementos Principais Representados

O diagrama evidencia os principais aspetos da modelação orientada a objetos aplicada no sistema:

Encapsulamento: Através da definição clara de propriedades privadas e métodos públicos para cada classe;

Herança: Mediante a utilização de classes base que fornecem funcionalidades comuns a múltiplas entidades;

Polimorfismo: Através da definição de interfaces que permitem múltiplas implementações;
Abstração: Pela definição de contratos que isolam as dependências entre camadas.

4.3 Valor para o Desenvolvimento

A elaboração deste diagrama abaixo representado pela Figura 1no Visual Paradigm permitiu validar a coerência do design antes da fase de implementação, identificando potenciais problemas de conceção e otimizando a estrutura de relações entre classes. Esta ferramenta servirá como documentação técnica de referência para manutenções futuras.



Figura 1 Diagrama de Classes UML do Sistema de Gestão de Alojamentos Turísticos

5. Modelação das Classes Principais

O sistema de gestão de alojamentos turísticos foi modelado com base em seis entidades principais que representam os conceitos fundamentais do domínio de negócio. Cada classe foi desenhada para encapsular dados e comportamentos específicos, seguindo os princípios de orientação a objetos.

5.1 Classes Principais Implementadas

Alojamento

A classe Alojamento constitui a entidade central do sistema, representando as unidades de hospedagem disponíveis para reserva. Esta classe inclui propriedades para identificação única, caracterização da propriedade (nome, tipo, localização), informações de capacidade e preço, bem como o estado atual de disponibilidade. Os métodos principais permitem verificar a disponibilidade temporal e calcular preços com base em períodos de estadia.

Cliente

A classe Cliente gere a informação dos hóspedes, armazenando dados pessoais, contactos e documentos de identificação. Para além das propriedades básicas, esta classe oferece funcionalidades de validação de dados.

Reserva

A classe Reserva estabelece a relação entre clientes e alojamentos, coordenando todo o processo de marcação. Esta entidade gere informações sobre períodos de estadia, número de hóspedes, estados do ciclo de vida da reserva e cálculos financeiros.

CheckIn

A classe CheckIn representa o processo de entrada dos clientes nos alojamentos, registando informações específicas da estadia como data e hora de entrada, hóspedes adicionais e observações relevantes. Esta classe funciona como uma extensão da reserva, capturando dados operacionais do momento do check-in.

Registo

A classe Registo atua como agregadora de operações de criação e atualização de entidades no sistema, fornecendo uma camada de abstração para operações de escrita. Esta classe centraliza a lógica comum a todas as operações de persistência, garantindo consistência e rastreabilidade.

Consulta

A classe Consulta especializa-se em operações de pesquisa e filtragem de dados, oferecendo funcionalidades avançadas para recuperação de informação.

User

A classe User é a entidade fundamental para a segurança e controlo de acessos à aplicação. Armazena as credenciais de autenticação, como o nome de utilizador e a palavra-passe. Esta classe permite distinguir os diferentes níveis de permissão no sistema, garantindo que apenas utilizadores autorizados possam realizar operações críticas.

Employee

A classe Employee (Funcionário) representa o capital humano responsável pela operação da unidade turística. Ao contrário do Cliente, esta classe foca-se na identificação profissional e na responsabilidade operacional. Contém atributos como número de identificador, nome, cargo e NIF. Esta entidade é crucial para a rastreabilidade do sistema, uma vez que o seu identificador é vinculado a cada operação de Check-in, Check-out ou alteração de estado através da classe de Registo, permitindo saber exatamente quem realizou cada ação.

5.2 Relações e Colaborações

As classes principais estabelecem entre si relações que refletem as interações do mundo real: os clientes efetuam reservas em alojamentos, as reservas dão origem a check-ins. Esta modelação permite uma representação fiel dos processos de negócio enquanto mantém uma separação clara de responsabilidades.

A modelação proposta fornece uma base sólida para a implementação do sistema, capturando de forma abrangente os requisitos do domínio de gestão de alojamentos turísticos. A estrutura definida suporta eficazmente os processos operacionais enquanto mantém a flexibilidade necessária para evoluções futuras do sistema.

6. Estratégia de Qualidade e Robustez

A garantia de qualidade e a robustez do sistema são asseguradas através de uma abordagem integrada que combina testes automatizados, tratamento proativo de exceções e um sistema de logs abrangente.

Será implementada uma suite de testes unitários com o objetivo de validar o comportamento isolado de cada componente, visando atingir a cobertura mínima de 50% exigida para o projeto. Estes testes são cruciais para a prevenção de regressões e para a validação contínua da lógica de negócio.

Para garantir a fiabilidade da aplicação, o tratamento de erros será baseado no desenvolvimento de exceções personalizadas e específicas do domínio. Esta abordagem permite uma gestão granular e contextualizada de falhas de negócio, facilitando a recuperação e a notificação adequada.

Complementarmente, será implementado um sistema de logging estruturado que irá registar operações críticas, eventos do sistema e, fundamentalmente, as exceções tratadas. Esta integração assegura que, em caso de falha, toda a informação relevante seja preservada no log para análise posterior e debugging.

Esta estratégia tripla (testes, exceções e logging) proporciona uma base sólida para o desenvolvimento de uma aplicação fiável e manutenível, capaz de lidar eficazmente com cenários de erro e de fornecer o rasto de auditoria necessário para a sua sustentação.

7. Estratégia de Persistência

Para o sistema de gestão de alojamentos turísticos, optou-se pela utilização de ficheiros JSON (JavaScript Object Notation) como mecanismo principal de persistência de dados. Esta decisão foi estratégica e fundamentada nas características específicas do projeto, nomeadamente o seu âmbito de pequena a média dimensão e a necessidade de flexibilidade na fase inicial de desenvolvimento.

O JSON constitui um formato de intercâmbio de dados leve, caracterizado pela sua fácil legibilidade e escrita, tanto para humanos quanto para máquinas. A sua estrutura baseada em pares chave-valor e arrays é inerentemente adequada para representar as entidades de domínio do sistema, como propriedades, clientes, reservas e transações, mantendo a estrutura dos objetos de forma direta.

7.1 Vantagens Técnicas

A seleção do formato JSON para a persistência inicial justifica-se pelas seguintes vantagens:

Simplicidade de Implementação: Permite uma implementação rápida e direta, ideal para a fase de prototipagem, evitando a sobrecarga de configuração de um servidor de base de dados relacional.

Portabilidade: Os ficheiros são facilmente transferíveis entre diferentes ambientes e sistemas, facilitando operações de backup, migração e partilha de dados.

Facilidade de Depuração (Debugging): A natureza legível do formato JSON permite a inspeção manual direta dos dados armazenados, simplificando significativamente o processo de desenvolvimento e resolução de erros.

Compatibilidade Nativa: O framework .NET oferece suporte nativo de alto desempenho para serialização e desserialização JSON através do namespace System.Text.Json, garantindo uma integração eficiente.

7.2. Otimizações e Considerações Críticas

A implementação do JSON como camada de persistência considerou fatores críticos para mitigar as limitações de ficheiros planos.

Otimizações Implementadas

Serialização Assíncrona: Todas as operações de leitura e escrita são realizadas de forma assíncrona para evitar bloqueios na interface de utilizador.

Gestão de Memória: Os dados são carregados para memória apenas quando necessário, reduzindo o consumo de recursos.

Validação de Integridade: Implementação de mecanismos de validação para garantir a integridade dos dados durante as operações de serialização.

A escolha do formato JSON para persistência de dados demonstra-se coerente e adequada para os objetivos do projeto na sua fase de desenvolvimento, oferecendo um equilíbrio ideal entre simplicidade e flexibilidade. A arquitetura implementada, baseada no Padrão Repository, assegura, contudo, a facilidade de evolução futura para sistemas de persistência mais robustos, mantendo a coesão e a qualidade da solução.

8. Enquadramento Arquitetural: Padrão Model-View-ViewModel (MVVM)

Para a estruturação do sistema de gestão de alojamentos turísticos, foi adotado o padrão arquitetural Model-View-ViewModel (MVVM). Esta escolha é estratégica e fundamentada na necessidade de desenvolver uma aplicação altamente testável, manutenível e escalável. O MVVM promove uma separação de responsabilidades clara e estrita entre a interface do utilizador e a lógica de negócio, um requisito crítico para a qualidade do software.

8.1 Componentes e Responsabilidades

O padrão MVVM define três componentes principais com funções bem delimitadas:

Model: Representa o Domínio do sistema, contendo as entidades de negócio (Alojamento, Cliente, Reserva) e a lógica de negócio essencial.

View: É responsável exclusiva pela apresentação da Interface do Utilizador (UI). É implementada em XAML e não deve conter lógica de aplicação (code-behind), exceto aquela relacionada diretamente com a manipulação da interface.

ViewModel: Atua como intermediário entre a View e o Model. Contém a lógica de apresentação e expõe os dados e comandos necessários para a View através de mecanismos de binding.

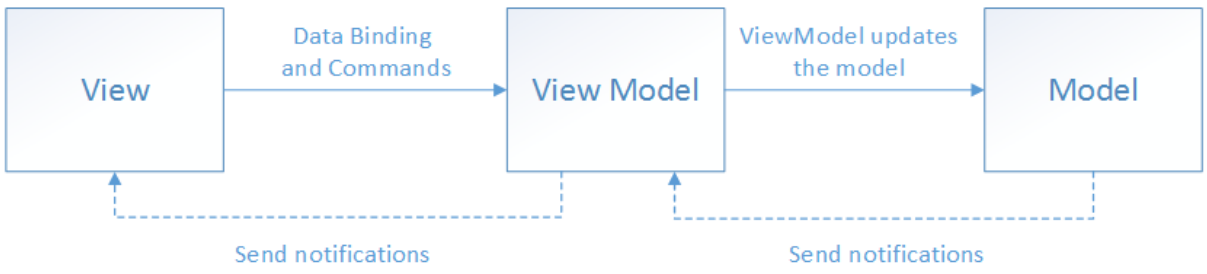


Figura 2 Representação de MVVM

8.2 Vantagens Estratégicas e Técnicas do MVVM

A adoção do MVVM confere vantagens significativas ao projeto, alinhadas com as boas práticas de engenharia de software.

Benefícios da Separação de Responsabilidades

A clara segregação de tarefas resulta em ganhos diretos:

Testabilidade Elevada: Os ViewModels podem ser sujeitos a testes unitários abrangentes, sem qualquer dependência da User Interface (UI).

Manutenibilidade Simplificada: Alterações e evoluções na View (Design) não impactam a lógica de negócio (Model) nem a lógica de apresentação (ViewModel).

Reutilização: Os ViewModels podem ser facilmente partilhados e reutilizados por diferentes Views, ou mesmo em diferentes tecnologias de apresentação.

Sinergia com WPF (Windows Presentation Foundation)

O padrão MVVM demonstra uma sinergia excecional com a tecnologia WPF através de mecanismos nativos que facilitam o desenvolvimento:

Data Binding: Permite a ligação declarativa e automática de dados entre a View e o ViewModel.

Commands: Facilita a ligação de ações da UI (p. ex., clique de botão) a métodos no ViewModel sem necessidade de code-behind.

Templates: Suporta a definição de templates de visualização baseados no tipo de dado exposto pelo ViewModel.

8.3 Fluxo de Dados e Controlo

O fluxo de dados é bidirecional e controlado pelo *ViewModel*:

A *View* exibe e recolhe dados do utilizador, ligada ao *ViewModel* através de *binding*.

O *ViewModel* processa comandos e dados, atualizando o *Model*.

O *Model* notifica o *ViewModel* sobre alterações de estado.

As alterações propagam-se automaticamente de volta à *View* através do *binding*, garantindo a coerência da interface.

Conclusão Arquitetural

A escolha e aplicação rigorosa do padrão MVVM, demonstra ser a estratégia arquitetural ideal para o sistema de gestão de alojamentos turísticos. Esta abordagem garante o cumprimento dos requisitos de manutenibilidade, testabilidade e escalabilidade, fornecendo uma estrutura robusta e sustentável para o desenvolvimento contínuo do software.

9. Requisitos do Sistema

A especificação de requisitos é uma etapa fundamental no desenvolvimento de software, pois define as condições e capacidades que o sistema deve possuir para satisfazer as necessidades do negócio de gestão de alojamentos turísticos. No âmbito desta unidade curricular, os requisitos foram delineados para garantir que a aplicação não só cumpra as funções operacionais básicas, mas que o faça de forma robusta e escalável, aplicando rigorosamente o paradigma da Programação Orientada a Objetos (POO).

Nesta fase, os requisitos dividem-se em dois grandes grupos: os Funcionais, que descrevem o comportamento e as funcionalidades do sistema (o "o quê"), e os Não Funcionais, que estabelecem critérios de qualidade e restrições técnicas (o "como"). A correta definição destes elementos permitiu estruturar a lógica de serviços e a camada de persistência em JSON de forma coerente com a arquitetura MVVM adotada.

9.1 Requisitos funcionais (O que o sistema faz):

- Gestão de Reservas: O sistema deve permitir criar, atualizar, cancelar e listar reservas.
- Controlo de Check-in/Out: Deve ser possível realizar o check-in (alterando o estado do alojamento para "Ocupado") e o check-out (libertando o alojamento e enviando-o para "Manutenção").
- Motor de Busca de Disponibilidade: O sistema deve filtrar alojamentos que não tenham sobreposição de datas com reservas existentes.
- Auditoria (Logs): Cada operação crítica (criar reserva, mudar estado) deve gerar um registo automático para rastreabilidade.
- Segurança: O sistema deve validar credenciais de utilizador e garantir que passwords cumpram requisitos de complexidade através de uma biblioteca externa.

9.2 Requisitos Não Funcionais (Qualidade do sistema):

- Persistência: Os dados devem ser preservados em formato JSON para garantir portabilidade.
- Robustez: O sistema deve tratar erros de negócio através de exceções personalizadas (BusinessException), impedindo estados inválidos.
- Manutenibilidade: Utilização do padrão Repository para isolar a lógica de acesso a dados da lógica de negócio.
- Testabilidade: Pelo menos 50% do código lógico deve ser coberto por testes unitários isolados com recurso a Mocks

10. Exceções Personalizadas: A Classe BusinessException

No desenvolvimento deste sistema, optou-se por não utilizar apenas as exceções genéricas do .NET. Foi implementada a classe **BusinessException**, que permite uma gestão granular e contextualizada de falhas de negócio.

- **Objetivo:** Capturar violações das regras de negócio (ex: tentar apagar um alojamento com reservas ativas ou definir um preço negativo) e impedir que a aplicação entre num estado inválido.
- **Vantagem Técnica:** Ao lançar uma exceção específica, o sistema consegue separar erros técnicos (como falhas de leitura de ficheiro) de erros de lógica (violação de regras de alojamento), facilitando o *debugging* e a apresentação de mensagens claras ao utilizador.
- **Integração com Logs:** Sempre que uma BusinessException é disparada, o evento é capturado pelo sistema de logging estruturado, preservando a informação para análise posterior.

11. Validação de Segurança via DLL Externa

Para reforçar a segurança do sistema, especificamente na criação de novos utilizadores, foi integrada uma biblioteca externa (DLL), que contém a classe **PasswordValidator**.

- **Separação de Responsabilidades:** Seguindo os princípios da POO e do padrão MVVM, a lógica complexa de validação de segurança foi delegada a um componente externo. Isto permite que o AuthenticationService se foque na lógica de autenticação sem precisar de implementar internamente algoritmos de verificação de complexidade de caracteres.
- **Funcionamento:** Antes de persistir um novo utilizador no repositório, o serviço invoca o método estático PasswordValidator.Validar(password, out string mensagemErro).
- **Controlo de Fluxo:** Se a DLL indicar que a password é inválida, o sistema interrompe o processo de criação e utiliza a mensagemErro retornada pela biblioteca para informar o utilizador sobre o requisito em falta.
- **Profissionalismo e Reutilização:** A utilização de uma DLL demonstra a capacidade de integrar componentes de terceiros e sublinha a modularidade do projeto, um aspeto central na arquitetura de sistemas informáticos modernos

12. Conclusão e Trabalho Futuro

O presente trabalho resultou na conceção e desenvolvimento de um sistema de gestão de alojamentos turísticos, construído com base nos princípios da Programação Orientada a Objetos (POO) e estruturado sob o padrão arquitetural Model-View-ViewModel (MVVM). A arquitetura implementada assegura uma separação rigorosa de responsabilidades entre os Modelos, as *Views* e os *ViewModels*, promovendo a manutenibilidade e a testabilidade eficazes da aplicação. A utilização de WPF para a interface gráfica e de JSON para a persistência de dados demonstrou ser uma escolha apropriada para o âmbito e complexidade atuais do projeto.

A realização deste projeto proporcionou um conjunto de aprendizagens fundamentais e mais-valias técnicas que ultrapassam a mera escrita de código:

- **Domínio da Arquitetura de Software:** A aplicação do padrão MVVM permitiu compreender a importância do desacoplamento, facilitando a criação de um sistema onde a lógica de negócio é independente da interface.
- **Consolidação dos Pilares da POO:** A implementação prática de conceitos como herança, polimorfismo, encapsulamento e abstração permitiu criar um código mais reutilizável, organizado e próximo da realidade do domínio de negócio.
- **Qualidade e Rigor no Desenvolvimento:** A introdução de testes unitários com recurso a *Mocks* e a utilização de exceções personalizadas foram mais-valias críticas, garantindo que o sistema é resiliente a erros e validando a eficácia das regras de negócio.
- **Exploração de Tecnologias Modernas:** A utilização de LINQ para manipulação de coleções de dados e de bibliotecas externas para segurança (como o validador de passwords) permitiu desenvolver uma solução mais eficiente e alinhada com as práticas atuais da indústria.
- **Pensamento Estruturado e Gestão de Persistência:** A experiência de implementar o *Repository Pattern* em conjunto com a serialização JSON conferiu uma visão clara sobre como gerir o ciclo de vida dos dados de forma estruturada e segura.

Em suma, este trabalho permitiu converter conhecimentos teóricos em competências práticas de engenharia de software, resultando numa base sólida e escalável preparada para futuras expansões funcionais

12.1 Dificuldades Observadas

O desenvolvimento desta fase enfrentou desafios críticos, superados através de uma análise técnica aprofundada e da aplicação rigorosa de padrões de desenho de software. Os principais obstáculos identificados foram:

- **Lógica de Negócio e Disponibilidade:** A compreensão e tradução das regras de negócio específicas da gestão de turismo revelaram-se complexas, particularmente na implementação de algoritmos para verificar a sobreposição de datas em reservas e garantir a disponibilidade real dos alojamentos em tempo útil.
- **Desacoplamento e Injeção de Dependência:** A correta modelação do domínio para garantir o baixo acoplamento entre as camadas exigiu um esforço adicional na implementação de Interfaces e no padrão *Repository*. A transição de listas simples para uma arquitetura onde os serviços dependem de abstrações foi fundamental para a robustez do sistema.
- **Implementação de Testes Unitários:** A criação de uma suite de testes que fosse "real" e não meramente formal constituiu um desafio técnico significativo. Foi necessário dominar a utilização de *Mocks* (biblioteca Moq) para isolar a lógica de negócio da camada de persistência, garantindo que os testes validassem efetivamente as regras de negócio sem interferência de ficheiros externos.
- **Gestão de Estados e Persistência JSON:** A seleção da estratégia de persistência em JSON exigiu uma análise cuidada para garantir que a integridade dos dados fosse mantida durante operações de escrita assíncrona, evitando corrupção de dados em cenários de atualizações simultâneas.

12.2 Trabalho Futuro

Com a conclusão da lógica de negócio, persistência e validação por testes unitários na presente fase, o trabalho futuro concentrar-se-á na expansão da robustez e na experiência do utilizador. As metas delineadas para as próximas etapas de desenvolvimento incluem:

- Implementação da Interface Gráfica (WPF): Desenvolvimento completo das janelas e componentes visuais utilizando XAML, garantindo o data binding total com as ViewModels já estruturadas.
- Migração para Base de Dados Relacional: Evolução da camada de persistência, substituindo os ficheiros JSON por um sistema como SQL Server ou SQLite, para suportar maior volume de dados e operações concorrentes mais complexas.
- Módulo de Faturação e Pagamentos: Integração de funcionalidades para emissão de faturas e processamento de pagamentos, expandindo a utilidade do sistema para a gestão financeira.
- Expansão da Suite de Testes: Alargamento da cobertura de testes para incluir testes de integração e testes de interface (UI), garantindo que o fluxo entre View e ViewModel permanece íntegro.
- Integração com Plataformas Online: Desenvolvimento de APIs ou módulos de sincronização para ligar a disponibilidade do alojamento a plataformas externas de reserva.

12.3 GitHub Pessoal:

O trabalho realizado ao longo deste semestre irá ser depositado no ***Github*** e pode ser acedido através do seguinte link:

- https://github.com/hugo31545/Gestao-de-alojamentos-turisticos

13. Referências bibliográficas

Neste capítulo apresentam-se as fontes de informação consultadas e utilizadas ao longo do desenvolvimento do presente trabalho. As referências incluem materiais universitários, documentação técnica e conteúdos online que serviram de apoio à implementação, compreensão e fundamentação teórica do projeto.

Documentação técnica do .NET. (n.d.). *System.Text.Json Namespace*. Disponível em: https://learn.microsoft.com/dotnet

Microsoft. (n.d.). *Padrão arquitetural Model-View-ViewModel (MVVM)*. Disponível em: https://learn.microsoft.com/pt-pt/dotnet/architecture/maui/mvvm

Microsoft. (n.d.). *Designing the infrastructure persistence layer - Repository Pattern*. Disponível em: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design

Microsoft. (2004). *Distributed System Design using Command Pattern, MSMQ, and .NET*. Disponível em: https://learn.microsoft.com/en-us/archive/msdn-magazine/2004/september/distributed-system-design-using-command-pattern-msmq-and-net